

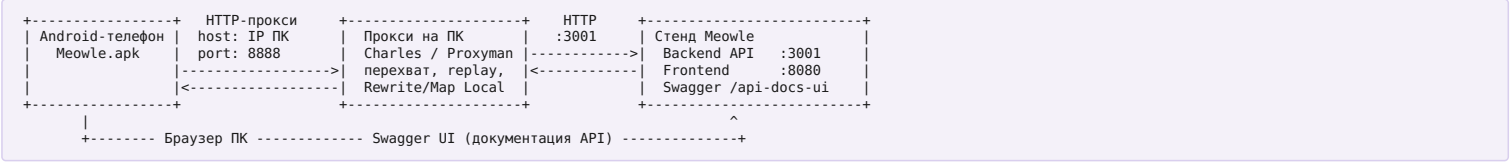
Лабораторная работа: Meowle Network Lab — отчёт

Тестирование мобильного приложения Meowle и API стенда через перехват трафика (Charles/Proxyman).

Стенд и инструменты

- Backend API: :3001 (в перехваченном трафике отвечал на 82.39.214.8:3001)
- Frontend: :8080; Swagger UI: /api-docs-ui/
- Мобильное приложение: Meowle-lab.apk, вход — имя «Алексей», телефон «+79990000000»
- Прокси: Charles/Proxyman (порт 8888), телефон и ПК в одной сети, ручной HTTP-прокси на телефоне
- Ручные запросы и replay — средствами прокси

Схема стенда



Телефон обращается к API только через прокси, поэтому весь трафик приложения доступен для просмотра и подмены. Swagger используется как карта доступных методов.

Часть 1. Таблица API из Swagger

Method	Path	Назначение	Параметры	Пример запроса	Пример ответа
POST	/cats/add	Добавление списка котов	Body: cats[] → {name*, gender, description}	{ "cats": [{ "name": "Барсик", "gender": "male" }] }	{ "cats": [{ "id": 1, "name": "Барсик", "likes": 0, "dislikes": 0 }] }
GET	/cats/get-by-id	Получение кота по id	Query: id* (number)	GET /cats/get-by-id?id=1	{ "cat": { "id": 1, "name": "Барсик", ... } }
POST	/cats/search	Поиск по имени и полу	Body: name* (string), gender (enum)	{ "name": "Бап", "gender": "male" }	{ "groups": [{ "title": "Б", "count": 1, "cats": [...] }] }
GET	/cats/search-pattern	Поиск по началу имени	Query: name* (string), limit (number)	GET /cats/search-pattern?name=Бап&limit=10	{ "moreResults": false, "cats": [...] }
POST	/cats/save-description	Сохранение описания кота	Body: catId* (number), catDescription* (string)	{ "catId": 1, "catDescription": "Ласковы й" }	{ "id": 1, "name": "Барсик", "description": "Ласковый" }
GET	/cats/validation	Правила валидации (regex)	—	GET /cats/validation	{ "id": 1, "description": "Только буквы", "regex": "..." }
GET	/cats/all	Список всех котов (по группам)	Query: order* (asc desc), gender (enum)	GET /cats/all?order=asc&gender=female	{ "groups": [{ "title": "А", "count": 2, "cats": [...] }] }
POST	/cats/{catId}/upload	Загрузка изображения кота	Path: catId* (int); form-data: file (binary)	POST /cats/1/upload (file=@cat.jpg)	{ "fileUrl": "/images/cat_1.jpg" }
GET	/cats/{catId}/photos	Получение фото по id кота	Path: catId* (int)	GET /cats/1/photos	{ "images": ["/images/cat_1.jpg"] }
GET	/version	Версия проекта	—	GET /version	{ "build": 123 }
POST	/cats/{catId}/like	Лайк коту	Path: catId* (int)	POST /cats/1/like	OK (text/plain)
DELETE	/cats/{catId}/like	Удаление лайка	Path: catId* (int)	DELETE /cats/1/like	OK
DELETE	/cats/{catId}/remove	Удаление кота	Path: catId* (int)	DELETE /cats/1/remove	OK
POST	/cats/{catId}/dislike	Дизлайк коту	Path: catId* (int)	POST /cats/1/dislike	OK
DELETE	/cats/{catId}/dislike	Удаление дизлайка	Path: catId* (int)	DELETE /cats/1/dislike	OK
GET	/cats/likes-rating	ТОП-10 по лайкам	—	GET /cats/likes-rating	[{ "name": "Барсик", "likes": 42 }]
GET	/cats/dislikes-rating	ТОП-10 по дизлайкам	—	GET /cats/dislikes-rating	[{ "name": "Музик", "dislikes": 17 }]
POST	/api/core/cats/search	Мобильный поиск котов	Body: name, order (asc desc), gender (enum)	{ "name": "Бап", "order": "asc" }	{ "groups": [...] }
POST	/api/core/cats/add	Мобильное добавление котов	Body: cats[] → {name, gender, description}	{ "cats": [{ "name": "Барсик", "gender": "male" }] }	Коты либо ошибка в errorDescription
GET	/api/core/cats/get-by-id	Мобильное получение по id	Query: id* (number)	GET /api/core/cats/get-by-id?id=1	Объект cat
POST	/api/core/cats/save-description	Мобильное сохранение описания	Body: catId (number), catDescription (string)	{ "catId": 1, "catDescription": "Ласковы й" }	Обновлённый cat
GET	/api/likes/cats/rating	Мобильный рейтинг	—	GET /api/likes/cats/rating	Объект с массивами likes и dislikes
POST	/api/likes/cats/{catId}/likes	Мобильное голосование	Path: catId* (int); Body: like (bool), dislike (bool)	POST /api/likes/cats/1/likes { "like": true }	Обновлённый cat
GET	/api/photos/cats/{catId}/photos	Мобильный список фото	Path: catId* (int)	GET /api/photos/cats/1/photos	Массив ссылок images
POST	/api/photos/cats/{catId}/upload	Мобильная загрузка фото	Path: catId* (int); form-data: file (binary)	POST /api/photos/cats/1/upload	Относительная ссылка fileUrl

Часть 2. Карта действий приложения

Действие	Method	URL	Request body	Code	Response (сокр.)	Комментарий
Поиск котика	POST	/api/core/cats/search	{ "gender": null, "name": "Барсик", "order": "asc" }	200	{ "groups": [{ "title": "Б", "cats": [{ "id": 1, "name": "Барсик" }], "count": 1 }] }	Мобильный эндпоинт. Результат сгруппирован по первой букве имени; count = числу котов в группе
Открытие карточки	GET	/api/core/cats/get-by-id?id=1	—	200	{ "cat": { "id": 1, "name": "Барсик", ..., "likes": 0, "dislikes": 0 } }	Объект завернут в ключ cat; данные совпадают с поиском
Лайк	POST	/api/likes/cats/1/likes	{ "dislike": false, "like": true }	200	{ "id": 1, ..., "likes": 1, "dislikes": 0 }	likes 0 → 1. Мобильный метод возвращает полный объект кота (web-метод /cats/{id}/like — text/plain «OK»)
Дизлайк	POST	/api/likes/cats/1/likes	{ "dislike": true, "like": false }	200	{ "id": 1, ..., "likes": 1, "dislikes": 1 }	Тот же эндпоинт с флагом dislike. dislikes 0 → 1, likes не сбросился

						(остался 1)
Добавление котика	POST	/api/core/cats/add	{ "cats": [{"description":"легенда","gender":"male","name":"лев"}]}	200	{ "cats": [{"id":7,"name":"Лев",..., "likes":0, "dislikes":0}]}	Имя нормализуется к заглавной: «лев» → «Лев». Присвоен id 7
Изменение описания	POST	/api/core/cats/save-description	{ "catDescription":"Любит проверять границы апиapi", "catId":1}	200	{ "id":1, "name":"Барсик", "description":"Любит проверять границы апиapi", ...}	Описание перезаписано; лайки/ дизлайки сохранены
Загрузка изображения	POST	/api/photos/cats/1/upload	multipart/form-data: file (image/jpeg, 42 КБ)	200	{ "fileUrl":"/photos/image-1782502741264.jpg"}	Возвращён относительный путь с таймстампом в имени файла
Открытие рейтинга	GET	/api/likes/cats/rating	—	200	{ "likes": [{"id":1, "name":"Барсик", ...}], "dislikes": [...]}	Объект с двумя массивами — likes и dislikes

Часть 3. Изменение запросов

№	Исходный запрос	Изменённый запрос	Ответ	Состояние данных	Баг / security	Исправление
1	POST /api/likes/cats/7/likes	POST /api/likes/cats/77/likes	404	Нет	Не уязвимость. Кота 77 нет, 404 корректен. Лайк существующего кота разрешён логикой приложения	Не требуется (контроль доступа к чужим объектам — см. эксп. 3)
2	POST /api/likes/cats/7/likes	Тот же запрос ×10 (replay)	200	likes 2 → 12	Security. Лайк не идемпотентен, нет привязки голоса к пользователю — накрутка	Хранить голос по паре пользователь+кот; повторный запрос не меняет счётчик. Rate limiting
3	POST /api/core/cats/save-description	{ "catDescription":"Изменено чужим пользователем", "catId":1}	200	Описание изменилось	Security. Broken access control: сервер не проверяет владельца записи	Проверять права на объект; редактирование — только автору/ администратору, иначе 403
4	POST /api/core/cats/add	{ "cats": [{"name":"Барсик", "gender":"male", "description":"дубль"}]}	200	Не изменилось: { "errorDescription":"Такое значение уже существует"}	Не баг по сути: дубликат отклонён. Замечание — для ошибки возвращается 200	Возвращать 409 Conflict вместо 200 с ошибкой в теле
5.1	POST /api/core/cats/add	{ "cats": [{"name":"Тест", "gender":"dog"}]}	200	Создан кот, gender стал unixex	Баг валидации. Недопустимый enum не отклонён, а молча приведён к unixex	Валидировать gender по списку; при недопустимом значении 400, без тихой подмены
5.2	POST /api/core/cats/add	{ "cats": [{"name":"","gender":"dog"}]}	200	Не изменилось: { "errorDescription":"Имя не может быть пустым"}	Частично корректно: пустое имя отклонено. Невалидный gender не проверен — проверка остановилась на имени	Валидировать все поля и возвращать полный список ошибок; статус 400
5.3	POST /api/core/cats/save-description	{ "catId":1, "catDescription":"<script>alert(1)</script>"}	200	Описание сохранено как неэкранированный JS	Security. Stored XSS: ввод сохраняется без санитизации	Санитизировать ввод, экранировать вывод; ограничить символы по /cats/validation
6	DELETE /cats/7/remove (web-метод)	Тот же	404 + Boom	Кот удалён	Security + баг. Деструктивный web-метод доступен напрямую и меняет данные; статус не отражает факт изменения	Защитить web-методы аутентификацией/авторизацией; корректные статусы (200/204 при успехе)
7	GET /api/core/cats/get-by-id?id=1	GET ...?id=777	404	Нет	Не баг. 404 — корректная реакция на несуществующий id	Не требуется

Не проверено в этом прогоне: загрузка файла с неправильным расширением и с подменённым MIME type (часть 3, пункт 8). Для полноты выполняется отдельно через breakpoint на /api/photos/cats/{id}/upload. В выводы не включено, чтобы не приписывать непроверенное поведение.

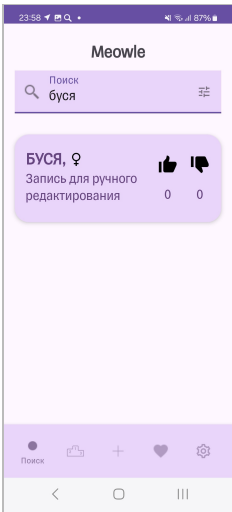
Часть 4. Изменение ответов (Charles Rewrite / Map Local)

№	Было	Стало	Инструмент
1	Имя «БУСЯ»	Имя «ВЗЛОМАНО»	Rewrite (Body, Response)
2	0 лайков	99999 лайков	Rewrite (Body, Response)
3	Реальный результат поиска (Кекс)	Подменённые коты (Призрак, Фейк)	Map Local
4	Загруженная фотография кота	Подменённый URL изображения	Rewrite / Map Local

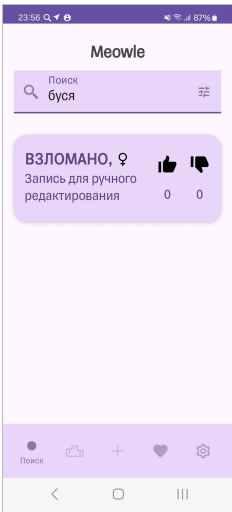
Скриншоты (до / после)

Эксперимент 1 — подмена имени в ответе поиска

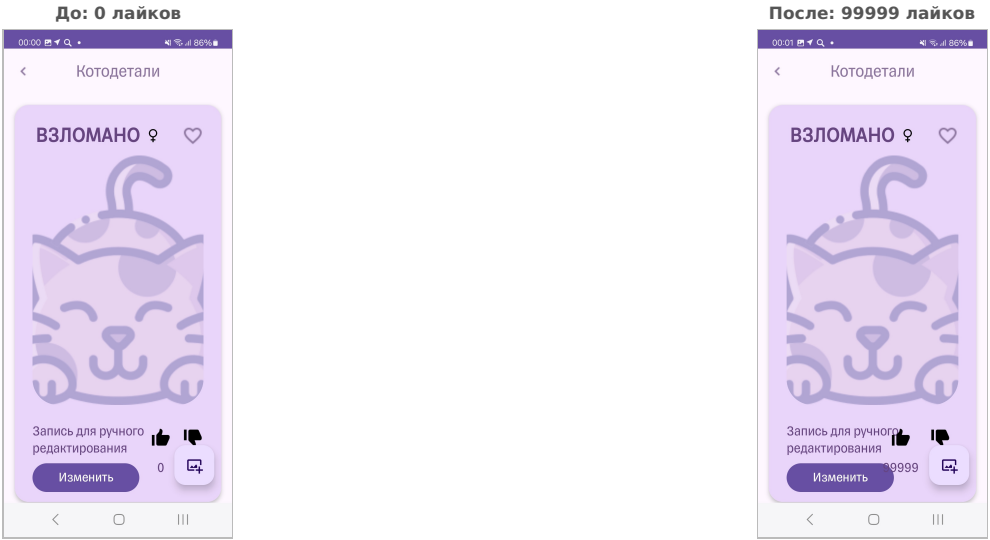
До: «БУСЯ»



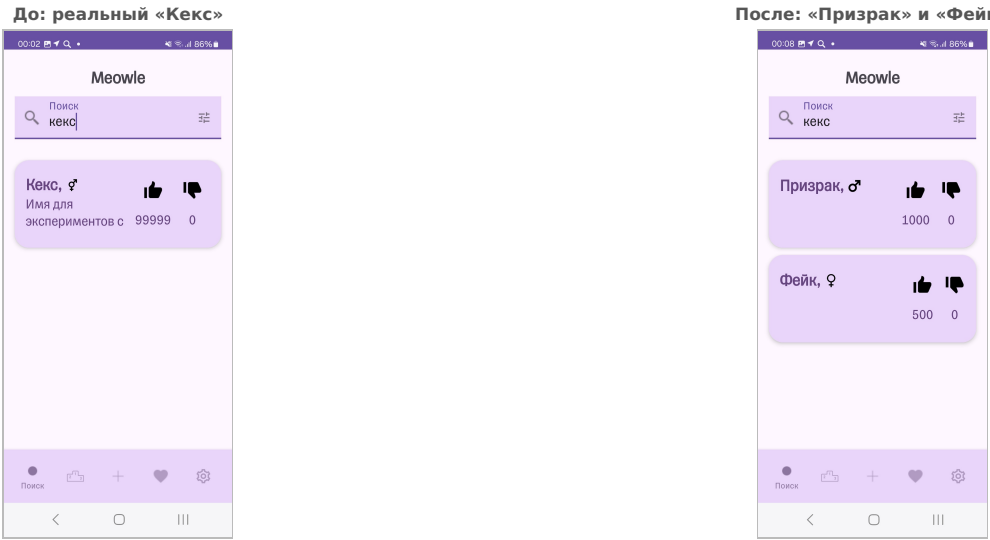
После: «ВЗЛОМАНО»



Эксперимент 2 — подмена количества лайков



Эксперимент 3 — подмена списка результатов поиска



Эксперимент 4 — подмена изображения



Что осталось на клиенте, а что сохранилось на сервере

Все четыре подмены затронули только клиент. Rewrite и Map Local меняют ответ на участке «сервер → прокси → приложение», уже после того как сервер сформировал настоящий ответ. Сервер этих изменений не получает, в базе ничего не меняется.

Проверка показывает это напрямую: после отключения правила Rewrite/Map Local и повторного запроса приложение снова показывает исходные данные (настоящее имя, реальные 0 лайков, реальный список котов и фото). Тот же эндпоинт, открытый в обход прокси, тоже возвращает оригинал.

Контраст с частью 3: там менялся **запрос**, и сервер, приняв его, записывал изменения в базу (лайки 2 → 12, перезапись описания, создание кота, удаление кота) — эти изменения сохранились. В части 4 менялся **ответ** — он не доходит до сервера и состояние не меняет. Вывод: подмена запроса может изменить данные на сервере; подмена ответа — нет.

Часть 5. Security-анализ

Finding 1. Изменение описания любого кота без проверки владельца

- **Severity: High**
- **Preconditions:** доступ к API через прокси/replay; известен catId.
- **Steps:** отправить POST /api/core/cats/save-description с {"catId":1,"catDescription":"Изменено чужим пользователем"} для чужой записи.
- **Actual:** 200 OK, описание перезаписано.
- **Expected:** сервер проверяет права на объект; чужую запись изменить нельзя (403).
- **Evidence:** часть 3, эксперимент 3.
- **Recommendation:** ввести владельца записи и серверную проверку прав на каждое изменение.

Finding 2. Накрутка голосов: лайк не идемпотентен и без rate limit

- **Severity: Medium**
- **Preconditions:** возможность повторять запрос (replay/Repeat).
- **Steps:** повторить POST /api/likes/cats/7/likes 10 раз подряд.
- **Actual:** каждый запрос увеличивает счётчик, likes 2 → 12.
- **Expected:** один пользователь = один голос; повторный запрос не меняет счётчик; ограничение частоты.
- **Evidence:** часть 3, эксперимент 2.
- **Recommendation:** хранить факт голоса по паре пользователь+кот, сделать операцию идемпотентной, добавить rate limiting.

Finding 3. Stored XSS в поле описания

- **Severity: High**
- **Preconditions:** доступ к save-description.
- **Steps:** отправить catDescription со значением <script>alert(1)</script>.
- **Actual:** 200 OK, значение сохранено без экранирования.
- **Expected:** ввод санитизируется/экранируется; опасная разметка не сохраняется или не исполняется при выводе.
- **Evidence:** часть 3, эксперимент 5.3.
- **Recommendation:** серверная санитизация ввода, экранирование вывода на клиенте, ограничение символов по /cats/validation.

Finding 4. Неаутентифицированный деструктивный web-эндпоинт

- **Severity: High**
- **Preconditions:** известен путь web-метода /cats/{id}/remove, не используемого мобильным сценарием.
- **Steps:** вызвать DELETE /cats/7/remove напрямую.
- **Actual:** кот удалён (состояние изменилось), ответ — 404 с Boom-payload.
- **Expected:** деструктивные операции — только при наличии прав; неиспользуемые web-методы закрыты или защищены.
- **Evidence:** часть 3, эксперимент 6.
- **Recommendation:** закрыть/защитить web-методы аутентификацией и авторизацией; разделить публичную и служебную поверхности API.

Finding 5. Слабая валидация входных данных (enum gender)

- **Severity: Medium**
- **Preconditions:** доступ к add.
- **Steps:** отправить {"cats":[{"name":"Тест","gender":"dog"}]}.
- **Actual:** 200 OK, недопустимый gender молча приведён к unisex.
- **Expected:** недопустимое значение enum отклоняется с 400 и описанием ошибки.
- **Evidence:** часть 3, эксперимент 5.1.
- **Recommendation:** строгая валидация по схеме; ошибка вместо тихой подмены; возврат списка ошибок по всем полям.

Finding 6. Расхождение между ограничениями UI и проверками на сервере

- **Severity: Medium**
- **Preconditions:** прямой доступ к API в обход интерфейса приложения.
- **Steps:** выполнять операции напрямую (произвольный catId, повторные голоса, web-методы), минуя экраны приложения.
- **Actual:** сервер выполняет действия, недоступные в интерфейсе; решения о допустимости фактически принимает клиент.
- **Expected:** все ограничения дублируются и проверяются на сервере независимо от клиента.
- **Evidence:** совокупно — часть 3 (эксперименты 2, 3, 6) и часть 4 (подмена ответов).
- **Recommendation:** перенести проверки прав, валидацию и бизнес-правила на сервер; не полагаться на ограничения интерфейса.

Баг-репорты

Bug 1. Лайк не идемпотентен — счётчик растёт при повторях

- **Severity: Medium**
- **Steps:** POST /api/likes/cats/7/likes повторить 10 раз.
- **Actual:** likes 2 → 12, каждый запрос засчитан.
- **Expected:** повторный голос того же пользователя не увеличивает счётчик.
- **Evidence:** часть 3, эксперимент 2.

Bug 2. Недопустимый gender молча приводится к unisex

- **Severity: Medium**
- **Steps:** POST /api/core/cats/add с gender:"dog".
- **Actual:** 200 OK, создан кот с gender:"unisex".

- **Expected:** 400 с ошибкой валидации, кот не создаётся.
- **Evidence:** часть 3, эксперимент 5.1.

Bug 3. /cats/{id}/remove возвращает 404, хотя данные затронуты

- **Severity: Medium**
- **Steps:** DELETE /cats/7/remove.
- **Actual:** кот удалён, но ответ — 404 Not Found с Boom-payload «Не найден id кота».
- **Expected:** статус соответствует результату: 200/204 при успешном удалении, 404 — только если объект отсутствует.
- **Evidence:** часть 3, эксперимент 6.

Bug 4. Описание сохраняется без экранирования спецсимволов

- **Severity: High**
- **Steps:** save-description со значением `<script>alert(1)</script>`.
- **Actual:** 200 OK, значение сохранено как есть.
- **Expected:** спецсимволы экранируются или ввод отклоняется по правилам валидации.
- **Evidence:** часть 3, эксперимент 5.3.

Bug 5. Бизнес-ошибка возвращается с кодом 200 OK

- **Severity: Low**
- **Steps:** добавить дубликат имени; либо отправить пустое имя.
- **Actual:** 200 OK с errorDescription в теле («Такое значение уже существует» / «Имя не может быть пустым»).
- **Expected:** ошибочные запросы получают статус 4xx (409 для дубликата, 400 для пустого имени).
- **Evidence:** часть 3, эксперименты 4 и 5.2.

Вывод

Мобильному клиенту нельзя доверять как источнику решений о допустимости действий. Перехват трафика показал, что приложение — это лишь интерфейс к API: те же операции выполняются напрямую, минуя экраны и ограничения интерфейса. Через прокси можно подставить произвольный catId, повторить голос десятки раз, перезаписать чужое описание, передать значение вне допустимого набора, сохранить неэкранированную разметку и вызвать web-метод, которого нет в мобильном сценарии.

Эксперименты с подменой ответов (часть 4) отделяют клиентскую видимость от серверного состояния: подмена ответа меняет только то, что видит приложение, и не сохраняется на сервере, тогда как подмена запроса (часть 3) меняет данные в базе. Отсюда практический критерий: значимы те проверки, что выполняются на сервере, а не в интерфейсе.

Сервер должен независимо от клиента: проверять права на объект перед изменением и удалением; валидировать все входные поля по схеме и отклонять недопустимые значения вместо тихой подмены; обеспечивать идемпотентность голосов и ограничение частоты запросов; санитизировать ввод и экранировать вывод; возвращать корректные HTTP-статусы; и не выставлять наружу неиспользуемые деструктивные методы без аутентификации и авторизации.